卷積神經網路 CNN 在影像辨識與超解析度上的應用

學號：411121033

姓名：邱國哲

完成日期：2025/6

一句話總結

CNN 在 Fashion-MNIST 與 CIFAR-10 資料集上分類表現，能有效提取空間特徵；Tiny Face 上實驗顯示，即使僅用 1/16 通道的輕量模型仍能保留關鍵特徵，證明在資源受限情境下具備實用潜力；其在多方向標籤影像處理的表現，展示出深度學習模型對於一般認定、最佳應用情境以外的資料型態的適應性與穩定性，可提供後續研究參考與改進方向。

<hr>

探索資料：

- Tiny Face Database
  - 來源：MIT 提供的 tiny 影像資料庫
  - 專案設定選擇了四個類別作為分類任務：人類、花卉、動物、建築
  - 這些類別具有顯著的視覺特徵，適合用來測試模型的特徵提取能力。
  - 資料集包含多樣的場景與物件，有助於模型學習穩定的特徵表示。

程式碼說明

```
import torch, torchvision
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, Subset
from torchvision.datasets import FashionMNIST, CIFAR10
from torchvision.transforms import ToTensor, Compose, Normalize, Resize, RandomHorizontalFlip, RandomCrop
import matplotlib.pyplot as plt
import numpy as np, time, os, random, pathlib, urllib.request, tarfile, shutil
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
from PIL import Image, ImageOps, ImageFilter
from torch.utils.data import Dataset
import glob, os, random, pathlib, urllib.request, tarfile, shutil
from PIL import Image
import torch, torchvision, time, os, random, pathlib
import numpy as np, matplotlib.pyplot as plt
from torchvision.datasets import FashionMNIST, CIFAR10
from torchvision.transforms import ToTensor, Compose, Normalize
from torch.utils.data import DataLoader, Subset
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns, pandas as pd
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)
if device.type == 'cuda':
    print('GPU:', torch.cuda.get_device_name(0))
```

程式碼說明：

- 主要的函式庫
  - 匯入 torch、torchvision、numpy、matplotlib、seaborn 等常用工具
  - 確保環境中安裝了必要的深度學習與資料處理套件
- 裝置設定
  - 自動偵測是否有 GPU，若有則使用 cuda，否則使用 cpu
  - 這樣設計可以在不同硬體環境下自動適應，提高程式的可移植性與效能表現。

```
# 資料集與類別設定
DATA_ROOT = pathlib.Path('./data')
CLASSES = ['Faces', 'Flowers', 'Animals', 'Buildings']
NUM_CLASSES = len(CLASSES)
print('Classes:', CLASSES)
```

程式碼說明：

- 定義資料根目錄
  - 設定資料集儲存路徑為 ./data
- 定義分類類別
  - 設定四個分類目標：Faces、Flowers、Animals、Buildings
  - 這四個類別涵蓋了日常生活中常見的視覺辨識任務，具備一定的多樣性與挑戰性。
- 顯示類別資訊
  - 透過 print 確認類別設定是否正確，方便後續除錯與檢查。

```
# 下載 Fashion-MNIST 與 CIFAR-10 資料集
tfm = Compose([ToTensor(), Normalize((0.5,), (0.5,))])
fm_tr = FashionMNIST(root='./data', train=True, download=True, transform=tfm)
fm_te = FashionMNIST(root='./data', train=False, download=True, transform=tfm)
cf_tr = CIFAR10(root='./data', train=True, download=True, transform=tfm)
cf_te = CIFAR10(root='./data', train=False, download=True, transform=tfm)
print('Fashion-MNIST train:', len(fm_tr), 'test:', len(fm_te))
print('CIFAR-10 train:', len(cf_tr), 'test:', len(cf_te))
```

程式碼說明：

- 資料前處理設定
  - 使用 ToTensor() 將影像轉換為 tensor
  - 使用 Normalize((0.5,), (0.5,)) 進行標準化，將像素值縮放至 [-1, 1] 範圍，有助於模型訓練穩定性與收斂速度。
- 下載 Fashion-MNIST 資料集
  - 訓練集：60,000 張灰階影像
  - 測試集：10,000 張灰階影像
  - 用於評估模型在簡單視覺任務上的表現。
- 下載 CIFAR-10 資料集
  - 訓練集：50,000 張彩色影像
  - 測試集：10,000 張彩色影像
  - 用於評估模型在較複雜、多樣性高的視覺任務上的表現。

```
# 建立 Tiny Face 資料集
ROOT = pathlib.Path('./tiny_face')
if not ROOT.exists():
    print('Tiny Face 資料夾不存在，請先執行下載與整理步驟')
else:
    for c in CLASSES:
        n = len(list((ROOT/'train'/c).glob('*')))
        print(f'{c}: {n} 張訓練影像')
```

程式碼說明：

- 設定 Tiny Face 資料夾路徑
  - 指定 ./tiny_face 為資料集根目錄
- 檢查資料夾是否存在
  - 若不存在，提示需先執行下載與整理步驟
- 統計各類別訓練影像數量
  - 遍歷 train/類別 資料夾，計算每個類別的影像數量
  - 確認資料是否完整，是後續訓練的基礎。

```
# 顯示 Fashion-MNIST 樣本
fig, axes = plt.subplots(2, 5, figure=fig_size, sharex=True, sharey=True)
for i, ax in enumerate(axes.flatten()):
    ax.imshow(fm_tr[i][0].squeeze(), cmap='gray')
    ax.axis('off')
    if i<5: ax.set_title(f'{fm_tr[i][1]}')
plt.suptitle('Fashion-MNIST 訓練樣本')
plt.tight_layout()
plt.savefig('fashion_samples.jpg', dpi=150)
plt.show()
```

程式碼說明：

- 建立 2×5 子圖
  - 共顯示 10 張訓練樣本，涵蓋 10 個類別
- 顯示灰階影像
  - 使用 cmap='gray' 呈現黑白視覺效果
- 加入類別標題
  - 每張圖上方顯示對應的類別編號
- 儲存與顯示
  - 輸出為 fashion_samples.jpg，解析度 150 dpi，方便後續報告使用。

程式碼說明：

- 顯示 CIFAR-10 樣本
  - 重複上述流程，顯示 10 張訓練樣本
- 觀察資料多樣性
  - 彩色影像包含更多顏色與紋理資訊，模型需要學習更穩定的特徵才能分類。

程式碼說明：

- 定義 DataLoader
  - 訓練集：batch_size=128，shuffle=True，drop_last=True
  - 測試集：batch_size=256，shuffle=False
  - 大 batch 有助於穩定梯度與加速訓練
- 設定 num_workers=2
  - 使用 2 個子進程平行讀取資料，提升效率
- 計算 steps_per_epoch
  - 用於後續學習率調度器的設定

```
# 簡單 CNN 模型
class SimpleCNN(nn.Module):
    def __init__(self, c=3, n=10):
        super().__init__()
        self.conv1 = nn.Conv2d(c, 32, 3, 1, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1, 1)
        self.fc1 = nn.Linear(64*8*8, 128)
        self.fc2 = nn.Linear(128, n)
    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2)
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)
        x = x.view(-1, 64*8*8)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Using device:', device)

def train(model, loader, epochs=1, lr=1e-3):
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    ce = nn.CrossEntropyLoss()
    for e in range(epochs):
        t0, loss_sum, n = time.time(), 0.0, 0
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            opt.zero_grad()
            loss = ce(model(x), y)
            loss.backward()
            opt.step()
            loss_sum += loss.item()*x.shape[0]
            n += x.shape[0]
        print(f'Epoch {e+1}/{epochs} loss={loss_sum/n:.4f} time={time.time()-t0:.1f}s')
# 訓練 Fashion-MNIST 模型
model = SimpleCNN(1, 10).to(device)
train(model, DataLoader(fm_tr, batch_size=128, shuffle=True, drop_last=True, num_workers=2), epochs=3)
# 測試 Fashion-MNIST 模型
model.eval()
with torch.no_grad():
    pred = torch.cat([model(x.to(device)).argmax(1).cpu() for x, _ in DataLoader(fm_te, batch_size=256)])
acc = (pred == fm_te.targets).mean()
print(f'Fashion-MNIST test accuracy: {acc:.4f}')
```

程式碼說明：

- 定義 SimpleCNN 模型
  - 輸入通道 c=3，輸出類別 n=10
  - 卷積層：conv1 (3→32)、conv2 (32→64)，皆使用 3×3 kernel
  - 全連接層：fc1 (64×8×8→128)、fc2 (128→10)
  - 使用 ReLU 與 max pooling 進行特徵提取與降維
- 訓練函數 train
  - 使用 Adam 優化器
  - 交叉熵損失函數
  - 記錄每 epoch 的 loss 與 time
- 訓練 Fashion-MNIST 模型
  - 3 個 epoch，batch_size=128
  - 使用 2 個 worker 加速資料讀取
- 測試 Fashion-MNIST 模型
  - 計算測試集準確率

```
# 訓練 CIFAR-10 模型
model = SimpleCNN(3, 10).to(device)
train(model, DataLoader(cf_tr, batch_size=128, shuffle=True, drop_last=True, num_workers=2), epochs=5)
# 測試 CIFAR-10 模型
model.eval()
with torch.no_grad():
    pred = torch.cat([model(x.to(device)).argmax(1).cpu() for x, _ in DataLoader(cf_te, batch_size=256)])
acc = (pred == cf_te.targets).mean()
print(f'CIFAR-10 test accuracy: {acc:.4f}')
```

程式碼說明：

- 訓練 CIFAR-10 模型
  - 5 個 epoch，batch_size=128
  - 使用 2 個 worker 加速資料讀取
- 測試 CIFAR-10 模型
  - 計算測試集準確率

```
# 下載 Tiny Face 資料集
import urllib.request, tarfile, shutil, pathlib
DATA = pathlib.Path('./data')
URL = 'http://vision.ucsd.edu/data/tiny_face_dataset.tar.gz'
if not (DATA/'tiny_face').exists():
    print('下載中...')
    urllib.request.urlretrieve(URL, DATA/'tiny_face.tar.gz')
    print('解壓縮中...')
    with tarfile.open(DATA/'tiny_face.tar.gz') as t:
        t.extractall(DATA)
    print('完成')
else:
    print('已存在')
```

程式碼說明：

- 設定資料來源
  - 使用 UCSD 提供的 Tiny Face 資料集
  - 檔案格式為 .tar.gz 壓縮檔
- 自動下載與解壓縮
  - 若資料夾不存在，自動從網路下載並解壓縮
  - 若已存在，則跳過以節省時間

```
# 整理 Tiny Face 為 train/類別 結構
import os, shutil, random, pathlib
SRC = pathlib.Path('./data/tiny_face')
DST = pathlib.Path('./tiny_face')
if not (DST/'train').exists():
    for c in CLASSES:
        (DST/'train'/c).mkdir(parents=True)
    for f in (SRC/'faces').glob('*'):
        shutil.copy(f, (DST/'train'/'Faces'/f.name))
    for f in (SRC/'flowers').glob('*'):
        shutil.copy(f, (DST/'train'/'Flowers'/f.name))
    for f in (SRC/'animals').glob('*'):
        shutil.copy(f, (DST/'train'/'Animals'/f.name))
    for f in (SRC/'buildings').glob('*'):
        shutil.copy(f, (DST/'train'/'Buildings'/f.name))
    print('完成')
else:
    print('已存在')
```

程式碼說明：

- 設定來源與目標路徑
  - 來源：./data/tiny_face
  - 目標：./tiny_face
- 建立 train/類別 結構
  - 為每個類別建立對應資料夾
- 複製檔案
  - 將原始資料依類別分類複製到目標資料夾
- 確認完成
  - 若已存在，則跳過以防重複操作

```
# 簡單 Tiny Face 模型
class TinyFaceCNN(nn.Module):
    def __init__(self, c=3, n=4):
        super().__init__()
        self.conv1 = nn.Conv2d(c, 16, 3, 1, 1)
        self.conv2 = nn.Conv2d(16, 32, 3, 1, 1)
        self.fc1 = nn.Linear(32*8*8, 64)
        self.fc2 = nn.Linear(64, n)
    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2)
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)
        x = x.view(-1, 32*8*8)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
# 訓練 Tiny Face 模型
model = TinyFaceCNN(3, 4).to(device)
train(model, DataLoader(tf_tr, batch_size=128, shuffle=True, drop_last=True, num_workers=2), epochs=10)
# 測試 Tiny Face 模型
model.eval()
with torch.no_grad():
    pred = torch.cat([model(x.to(device)).argmax(1).cpu() for x, _ in DataLoader(tf_te, batch_size=256)])
acc = (pred == tf_te.targets).mean()
print(f'Tiny Face test accuracy: {acc:.4f}')
```

程式碼說明：

- 定義 TinyFaceCNN 模型
  - 輸入通道 c=3，輸出類別 n=4
  - 卷積層：conv1 (3→16)、conv2 (16→32)，皆使用 3×3 kernel
  - 全連接層：fc1 (32×8×8→64)、fc2 (64→4)
  - 使用 ReLU 與 max pooling 進行特徵提取與降維
- 訓練 Tiny Face 模型
  - 10 個 epoch，batch_size=128
  - 使用 2 個 worker 加速資料讀取
- 測試 Tiny Face